

Experiment-5 Construct a lexical analyzer [using c code]

1. AIM

Identify the tokens from simple statement as input stored in a linear array

ALGORITHM

- ❖ Read the input statement into a linear character array.
- ❖ Initialize index *i* to scan the array from left to right.
- ❖ If the current character is alphabetic, extract alphanumeric characters to form a token and check whether it is a keyword or identifier.
- ❖ If the current character is a digit, extract digits (and at most one dot) to form a numeric token.
- ❖ If the current character is a double quote, extract characters until the closing quote to form a string literal.
- ❖ If the current character is an operator symbol, check for single or double character operators and classify it.
- ❖ If the current character is a special symbol (; , () { }), classify it accordingly.
- ❖ Ignore whitespace or unrecognized characters and continue scanning.
- ❖ Repeat until the end of the input string is reached.
- ❖ Ask the user whether to continue and repeat the process if required.

CODE

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main()
{
    char input[200], token[100];
    int i, j;
    int ch = 1;

    while(ch == 1)
    {
        i = 0;

        printf("\nEnter the statement:\n");
        fgets(input, sizeof(input), stdin);

        while(input[i] != '\0')
        {
            // Keyword or Identifier
            if(isalpha(input[i]))
            {
                j = 0;
                while(isalnum(input[i]))
                {
```

```

        token[j++] = input[i++];
    }
    token[j] = '\0';

    if(strcmp(token,"int")==0 || strcmp(token,"float")==0 ||
        strcmp(token,"if")==0 || strcmp(token,"else")==0 ||
        strcmp(token,"while")==0 || strcmp(token,"return")==0 ||
        strcmp(token,"printf")==0)
        printf("Keyword      : %s\n", token);
    else
        printf("Identifier   : %s\n", token);
}

// Number (Integer or Float)
else if(isdigit(input[i]))
{
    j = 0;
    int dotCount = 0;

    while(isdigit(input[i]) || input[i] == '.')
    {
        if(input[i] == '.')
            dotCount++;

        token[j++] = input[i++];
    }
    token[j] = '\0';

    if(dotCount <= 1)
        printf("Number      : %s\n", token);
}

// String Literal
else if(input[i] == '"')
{
    j = 0;
    i++; // skip opening quote

    while(input[i] != '"' && input[i] != '\0')
    {
        token[j++] = input[i++];
    }
    token[j] = '\0';
    i++; // skip closing quote

    printf("String Literal : \"%s\"\n", token);
}

// Operator
else if(
    input[i]=='+' || input[i]=='-' ||
    input[i]=='*' || input[i]=='/' ||

```

```

        input[i]=='=' || input[i]=='<' ||
        input[i]=='>' || input[i]=='!'
    )
    {
        if(
            (input[i]=='+' && input[i+1]=='+') ||
            (input[i]=='-' && input[i+1]=='-') ||
            (input[i]=='=' && input[i+1]=='=') ||
            (input[i]=='<' && input[i+1]=='=') ||
            (input[i]=='>' && input[i+1]=='=') ||
            (input[i]=='!' && input[i+1]=='=')
        )
        {
            printf("Operator      : %c%c\n", input[i], input[i+1]);
            i += 2;
        }
        else
        {
            printf("Operator      : %c\n", input[i]);
            i++;
        }
    }

    // Special Symbol
    else if(
        input[i]==';' || input[i]==',' ||
        input[i]=='(' || input[i]==')' ||
        input[i]=='{' || input[i]=='}'
    )
    {
        printf("Special Symbol : %c\n", input[i]);
        i++;
    }
    else
        i++;
}

printf("\nDo you want to continue? (1/0): ");
scanf("%d", &ch);
getchar(); // clear newline
}

return 0;
}

```

OUTPUT

```
ct Clas C:\Users\OSlab\Desktop\gg\1 X + v
Enter the statement:
while(i<0){printf("hi");
Keyword      : while
Special Symbol : (
Identifier    : i
Operator     : <
Number       : 0
Special Symbol : )
Special Symbol : {
Keyword      : printf
Special Symbol : (
Identifier    : hi
Special Symbol : )
Special Symbol : ;

Do you want to continue? (1/0): 1

Enter the statement:
a=a+b+c;
Identifier    : a
Operator      : =
Identifier    : a
Operator      : +
Identifier    : b
Operator      : +
Identifier    : c
Special Symbol : ;

Do you want to continue? (1/0): |
```

CONCLUSION

The program performs lexical analysis by scanning a statement stored in a linear array and classifying its tokens into keywords, identifiers, numbers, string literals, operators, and special symbols.

2. AIM

Identify the tokens from small program (not exceeding 5 lines) as input stored in a text file

ALGORITHM

- ❖ Open the file in read mode and check whether it exists.
- ❖ Read one character at a time using `fgetc()` until end of file.
- ❖ If the character is alphabetic, collect consecutive alphanumeric characters to form a token.
- ❖ Compare the collected token with predefined keywords to classify it as keyword or identifier.
- ❖ If the character is a digit, collect digits (and optional dot) to form a numeric token.
- ❖ If the character is a double quote, read characters until the closing quote to form a string literal.
- ❖ If the character is an operator symbol, check for single or double-character operators and classify it.
- ❖ If the character is a special symbol (`( ) { } ; ,`), classify it accordingly.
- ❖ Use `ungetc()` when needed to push back extra characters into the file stream.
- ❖ Close the file after processing all characters.

SOURCE CODE

```

#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main()
{
    FILE *fp;
    char ch, token[100];
    int i;

    fp = fopen("sample.txt", "r");

    if(fp == NULL)
    {
        printf("File not found\n");
        return 0;
    }

    while((ch = fgetc(fp)) != EOF)
    {
        /* Keyword or Identifier */
        if(isalpha(ch))
        {
            i = 0;
            while(isalpha(ch) || isdigit(ch))
            {
                token[i++] = ch;
                ch = fgetc(fp);
            }
            token[i] = '\0';

            if(strcmp(token,"int")==0 || strcmp(token,"float")==0 ||
               strcmp(token,"while")==0 || strcmp(token,"if")==0 ||
               strcmp(token,"else")==0 || strcmp(token,"return")==0 ||
               strcmp(token,"printf")==0)
                printf("Keyword      : %s\n", token);
            else
                printf("Identifier   : %s\n", token);

            ungetc(ch, fp);
        }

        /* Number (integer or float) */
        else if(isdigit(ch))
        {
            i = 0;
            int dotCount = 0;

            while(isdigit(ch) || ch == '.')
            {
                if(ch == '.')
                    dotCount++;
            }
        }
    }
}

```

```

        token[i++] = ch;
        ch = fgetc(fp);
    }
    token[i] = '\0';

    printf("Number      : %s\n", token);
    ungetc(ch, fp);
}

/* String Literal */
else if(ch == '"')
{
    i = 0;
    token[i++] = ch;

    ch = fgetc(fp);
    while(ch != '"' && ch != EOF)
    {
        token[i++] = ch;
        ch = fgetc(fp);
    }

    token[i++] = '"';
    token[i] = '\0';

    printf("String      : %s\n", token);
}

/* Operators */
else if(ch=='+' || ch=='-' || ch=='*' || ch=='/' ||
        ch=='=' || ch=='<' || ch=='>' || ch=='!')
{
    char next = fgetc(fp);

    if((ch=='+' && next=='+') || (ch=='-' && next=='-') ||
        (ch=='=' && next=='=') || (ch=='<' && next=='<') ||
        (ch=='>' && next=='>') || (ch=='!' && next=='='))
    {
        printf("Operator    : %c%c\n", ch, next);
    }
    else
    {
        printf("Operator    : %c\n", ch);
        ungetc(next, fp);
    }
}

/* Special Symbols */
else if(ch=='(' || ch==')' || ch=='{' || ch=='}' ||
        ch==',' || ch==';')
{

```

```

        printf("Special Symbol : %c\n", ch);
    }
}

fclose(fp);
return 0;
}

```

OUTPUT

```

C:\Users\OSlab\Desktop\gg\2 x + v
Keyword      : int
Identifier    : a
Operator      : =
Number        : 10
Special Symbol : ;
Keyword      : int
Identifier    : b
Operator      : =
Number        : 0.1
Special Symbol : ;
Keyword      : while
Special Symbol : (
Identifier    : a
Operator      : >
Number        : 0
Special Symbol : )
Special Symbol : {
Identifier    : a
Operator      : =
Identifier    : a
Operator      : -
Identifier    : b
Special Symbol : ;
Identifier    : a
Operator      : --
Special Symbol : ;
Keyword      : printf
Special Symbol : (
String        : "hello world"
Special Symbol : )
Special Symbol : ;
Special Symbol : }

-----
Process exited after 0.03941 seconds with return value 0
Press any key to continue . . . |

```

CONCLUSION

The program performs lexical analysis on a file by reading characters sequentially and classifying them into keywords, identifiers, numbers, strings, operators, and special symbols.

3. AIM

Identify the tokens from small program (not exceeding 5 lines) as input get it from the user and store it in a text file

ALGORITHM

- ❖ Open a text file in write mode to store the user's input program.
- ❖ Accept characters from the user (maximum 5 lines) and write them into the file.

- ❖ Close the file after storing the input.
 - ❖ Reopen the same file in read mode.
 - ❖ Read characters one by one until end of file.
 - ❖ If a character is alphabetic, collect consecutive alphanumeric characters to form a token.
 - ❖ Compare the token with predefined keywords to classify it as keyword or identifier.
 - ❖ If a character is a digit, collect digits (and optional dot) to form a numeric token.
 - ❖ If a character is " , collect characters until the closing quote to form a string literal.
 - ❖ If a character is an operator symbol, check for single or double-character operators and classify it.
 - ❖ If a character is a special symbol (() { } ; ,) , classify it accordingly.
- Use `ungetc ( )` when required to push back extra characters into the file stream.
- ❖ Close the file after tokenization is complete.

SOURCE CODE

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main()
{
    FILE *fp;
    char ch, token[100];
    int i, lines = 0;

    /* STEP 1: Get input from user and store in file */
    fp = fopen("input.txt", "w");
    if(fp == NULL)
    {
        printf("File error\n");
        return 0;
    }

    printf("Enter a small program:\n");

    while(lines < 5)
    {
        ch = getchar();
        fputc(ch, fp);

        if(ch == '\n')
            lines++;

        if(ch == EOF)
            break;
    }
}
```



```

fclose(fp);

/* STEP 2: Read file and identify tokens */
fp = fopen("input.txt", "r");
if(fp == NULL)
{
    printf("File not found\n");
    return 0;
}

printf("\n--- TOKENS IDENTIFIED ---\n");

while((ch = fgetc(fp)) != EOF)
{
    /* Keyword or Identifier */
    if(isalpha(ch))
    {
        i = 0;
        while(isalpha(ch) || isdigit(ch))
        {
            token[i++] = ch;
            ch = fgetc(fp);
        }
        token[i] = '\0';

        if(strcmp(token,"int")==0 || strcmp(token,"float")==0 ||
            strcmp(token,"while")==0 || strcmp(token,"if")==0 ||
            strcmp(token,"else")==0 || strcmp(token,"return")==0 ||
            strcmp(token,"printf")==0)
            printf("Keyword      : %s\n", token);
        else
            printf("Identifier   : %s\n", token);

        ungetc(ch, fp);
    }

    /* Number (integer or float) */
    else if(isdigit(ch))
    {
        i = 0;
        while(isdigit(ch) || ch == '.')
        {
            token[i++] = ch;
            ch = fgetc(fp);
        }
        token[i] = '\0';

        printf("Number        : %s\n", token);
        ungetc(ch, fp);
    }

    /* String literal */

```

```

else if(ch == "")
{
    i = 0;
    token[i++] = ch;

    ch = fgetc(fp);
    while(ch != "" && ch != EOF)
    {
        token[i++] = ch;
        ch = fgetc(fp);
    }

    token[i++] = "";
    token[i] = '\0';

    printf("String      : %s\n", token);
}

/* Operators */
else if(ch=='+' || ch=='-' || ch=='*' || ch=='/' ||
        ch=='=' || ch=='<' || ch=='>' || ch=='!')
{
    char next = fgetc(fp);

    if((ch=='+' && next=='+') || (ch=='-' && next=='-') ||
        (ch=='=' && next=='=') || (ch=='<' && next=='=') ||
        (ch=='>' && next=='=') || (ch=='!' && next=='='))
    {
        printf("Operator      : %c%c\n", ch, next);
    }
    else
    {
        printf("Operator      : %c\n", ch);
        ungetc(next, fp);
    }
}

/* Special symbols */
else if(ch=='(' || ch==')' || ch=='{' || ch=='}' ||
        ch==';' || ch==',')
{
    printf("Special Symbol : %c\n", ch);
}
}

fclose(fp);
return 0;
}

```

OUTPUT

```
C:\Users\OSlab\Desktop\gg\3 X + v
Enter a small program:
for(int i=0;i<n;i++){
printf(i++);
printf(++i);
}

--- TOKENS IDENTIFIED ---
Identifier      : for
Special Symbol  : (
Keyword         : int
Identifier      : i
Operator        : =
Number          : 0
Special Symbol  : ;
Identifier      : i
Operator        : <
Identifier      : n
Special Symbol  : ;
Identifier      : i
Operator        : ++
Special Symbol  : )
Special Symbol  : {
Keyword         : printf
Special Symbol  : (
Identifier      : i
Operator        : ++
Special Symbol  : )
Special Symbol  : ;
Keyword         : printf
Special Symbol  : (
Operator        : ++
Identifier      : i
Special Symbol  : )
Special Symbol  : ;
Special Symbol  : }

-----
Process exited after 4.359 seconds with return value 0
Press any key to continue . . . |
```

CONCLUSION

The program stores a small user-entered program in a file and then performs lexical analysis to identify and classify its tokens systematically.

Experiment-6

AIM

Construct a lexical analyser using LEX tool.

ALGORITHM

- ❖ Define token patterns using regular expressions in the definition section.
- ❖ Write rules to match each pattern with appropriate token actions.
- ❖ Ignore whitespace characters during scanning.
- ❖ Print token type and matched text using `yytext`.
- ❖ Handle unmatched characters as unknown tokens.
- ❖ Call `yyllex()` in `main()` to start lexical analysis.
- ❖ Return 1 in `yywrap()` to indicate end of input.

SOURCE CODE

```
%{
#include <stdio.h>
}%

/* Definitions */
STRING    \"[^\"]*\"
FLOAT     [0-9]+\"[0-9]+
INT       [0-9]+
ID        [a-zA-Z_][a-zA-Z0-9_]*
INCDEC    \\+|\\+|\\-|-
RELOP     ==|!=|<=|>=
OP        [+\\-*/=]
BRACKET    [(\\)(\\[\\]\\{\\})
PUNCT     [,.;:!]

%%

{STRING}  { printf("String Literal : %s\\n", yytext); }
{FLOAT}   { printf("Decimal Number : %s\\n", yytext); }
{INT}     { printf("Integer Number : %s\\n", yytext); }
{ID}      { printf("Identifier : %s\\n", yytext); }

{INCDEC}  { printf("Inc/Dec Operator : %s\\n", yytext); }
{RELOP}   { printf("Relational Operator : %s\\n", yytext); }
{OP}      { printf("Operator : %s\\n", yytext); }

{BRACKET} { printf("Bracket : %s\\n", yytext); }
{PUNCT}   { printf("Punctuation : %s\\n", yytext); }

[ \\t\\n]+ ; /* Ignore whitespace */
.         { printf("Unknown : %s\\n", yytext); }

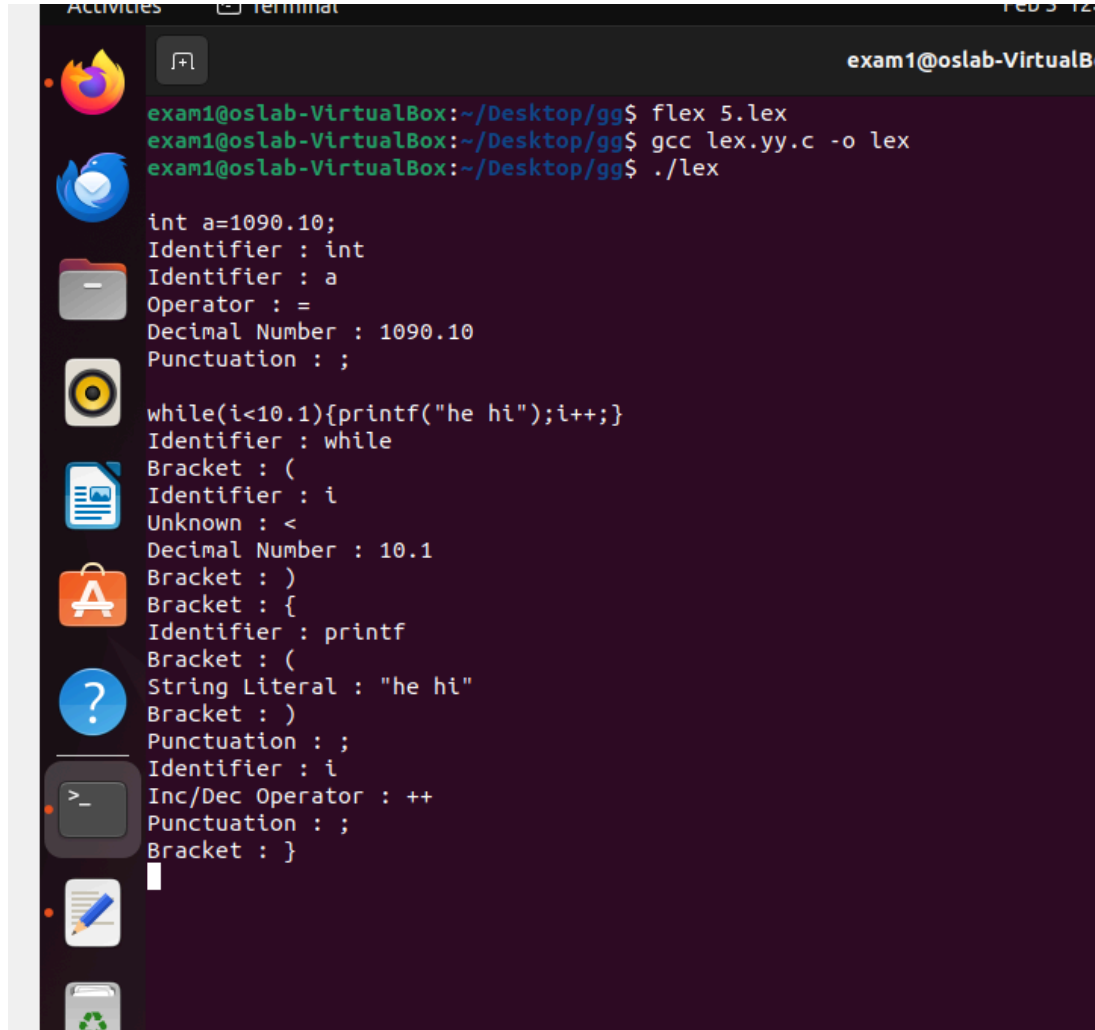
%%

int main() {
    yyllex();
    return 0;
}

int yywrap() {
    return 1;
}
```

}

OUTPUT



```
exam1@oslab-VirtualBox:~/Desktop/gg$ flex 5.lex
exam1@oslab-VirtualBox:~/Desktop/gg$ gcc lex.yy.c -o lex
exam1@oslab-VirtualBox:~/Desktop/gg$ ./lex

int a=1090.10;
Identifier : int
Identifier : a
Operator : =
Decimal Number : 1090.10
Punctuation : ;

while(i<10.1){printf("he hi");i++;}
Identifier : while
Bracket : (
Identifier : i
Unknown : <
Decimal Number : 10.1
Bracket : )
Bracket : {
Identifier : printf
Bracket : (
String Literal : "he hi"
Bracket : )
Punctuation : ;
Identifier : i
Inc/Dec Operator : ++
Punctuation : ;
Bracket : }
```

CONCLUSION

The LEX program automatically generates a lexical analyser that scans input using regular expressions and classifies tokens efficiently.

Experiment-7 Using LEX tool,

1. AIM

Implement LEX code to count the frequency of the given word in a file

ALGORITHM

- ❖ Declare a counter variable and a target word variable in the definition section.
- ❖ Accept the target word from the command-line argument.
- ❖ Use a regular expression to match alphabetic words from the input file.

- ❖ Compare each matched word (**yytext**) with the target word using **strcmp()**.
- ❖ Increment the counter if both words match exactly.
- ❖ Ignore all other characters including spaces and newlines.
- ❖ After scanning completes, display the frequency count.
- ❖ Return 1 in **yywrap()** to indicate end of file.

SOURCE CODE

```
%{
#include <stdio.h>
#include <string.h>

int count = 0;
char target[50];
}%

%%

[a-zA-Z]+ {
    if (strcmp(yytext, target) == 0)
        count++;
}

.\n    ; /* Ignore everything else */

%%

int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: ./wordcount <word>\n");
        return 1;
    }

    strcpy(target, argv[1]);

    yylex();

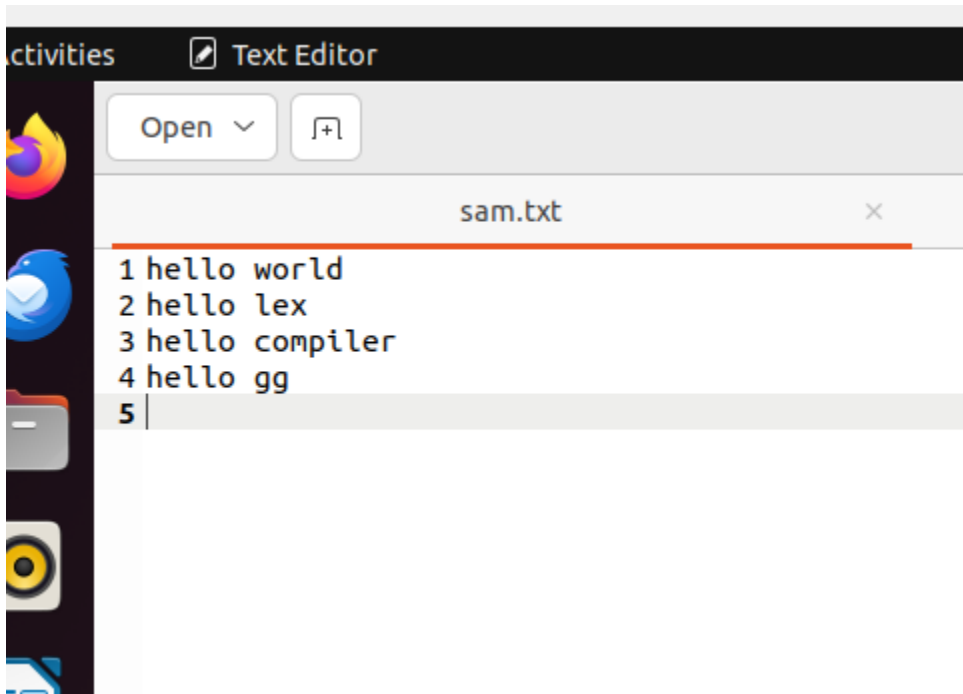
    printf("Frequency of word '%s' = %d\n", target, count);

    return 0;
}

int yywrap() {
    return 1;
}
```

OUTPUT

```
exam1@oslab-VirtualBox:~/Desktop/gg$  
exam1@oslab-VirtualBox:~/Desktop/gg$ flex word.l  
exam1@oslab-VirtualBox:~/Desktop/gg$ gcc lex.yy.c -o word  
exam1@oslab-VirtualBox:~/Desktop/gg$ ./word hello < sam.txt  
Frequency of word 'hello' = 4  
exam1@oslab-VirtualBox:~/Desktop/gg$ ./word gg < sam.txt  
Frequency of word 'gg' = 1  
exam1@oslab-VirtualBox:~/Desktop/gg$ S
```



CONCLUSION

The LEX program efficiently scans the input file and counts how many times a specified word appears using pattern matching and string comparison.

2. AIM

Implement LEX code to replace a word with another taking input from file.

ALGORITHM

- ❖ Declare two character arrays to store the old word and the new word.
- ❖ Accept the old word and new word from command-line arguments.
- ❖ Use a regular expression to match alphabetic words from the input file.
- ❖ Compare each matched word (yytext) with the old word using strcmp().
- ❖ If the word matches, print the new word instead.
- ❖ If the word does not match, print the original word.
- ❖ Print all other characters (including spaces and newlines) unchanged.
- ❖ Call yylex() to process the file until end of input.
- ❖ Return 1 in yywrap() to indicate completion.

SOURCE CODE

```
%{
#include <stdio.h>
#include <string.h>

char oldword[50], newword[50];
%}

%%

[a-zA-Z]+ {
    if (strcmp(yytext, oldword) == 0)
        printf("%s", newword);
    else
        printf("%s", yytext);
}

.\n    { printf("%s", yytext); }

%%

int main(int argc, char *argv[]) {

    if (argc != 3) {
        printf("Usage: ./replace <oldword> <newword>\n");
        return 1;
    }

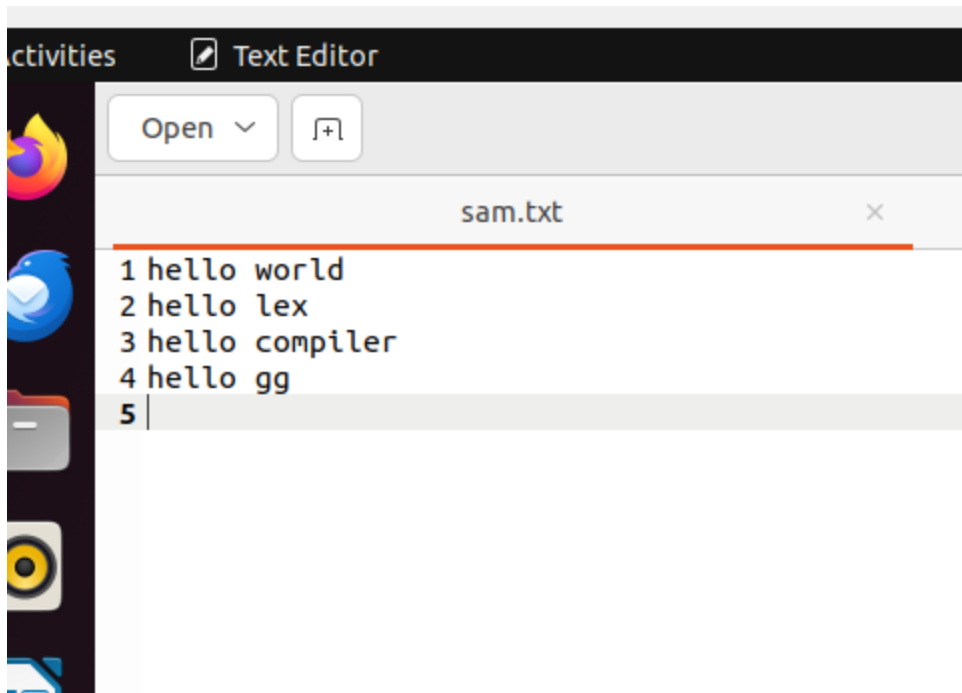
    strcpy(oldword, argv[1]);
    strcpy(newword, argv[2]);

    yylex();
    return 0;
}

int yywrap() {
    return 1;
}
```

OUTPUT

Before



```
exam1@oslab-VirtualBox: ~/Desktop/gg
exam1@oslab-VirtualBox:~/Desktop/gg$ flex replace.l
exam1@oslab-VirtualBox:~/Desktop/gg$ gcc lex.yy.c -o replace
exam1@oslab-VirtualBox:~/Desktop/gg$ ./replace hello Girija < sam.txt
Girija world
Girija lex
Girija compiler
Girija gg
exam1@oslab-VirtualBox:~/Desktop/gg$ ^C
exam1@oslab-VirtualBox:~/Desktop/gg$
```

And to replace in the actual file

```
./replace hello Girija < sam.txt > temp.txt
```

```
mv temp.txt sam.txt
```

CONCLUSION

The LEX program scans the input file and replaces every occurrence of a specified word with a new word using pattern matching and string comparison.

3. AIM

Implement LEX code to find the length of the longest word.

ALGORITHM

- ❖ Declare a variable to store the maximum word length.

- ❖ Use a regular expression to match alphabetic words from the input.
- ❖ For each matched word, calculate its length using `strlen(yytext)`.
- ❖ Compare the current word length with the stored maximum length.
- ❖ Update the maximum length if the current word is longer.
- ❖ Ignore all other characters including spaces and newlines.
- ❖ After scanning completes, print the maximum word length.
- ❖ Return 1 in `yywrap()` to indicate end of input.

SOURCE CODE

```
%{
#include <stdio.h>
#include <string.h>

int maxLen = 0;
%}

%%

[a-zA-Z]+ {
    int len = strlen(yytext);
    if (len > maxLen)
        maxLen = len;
}

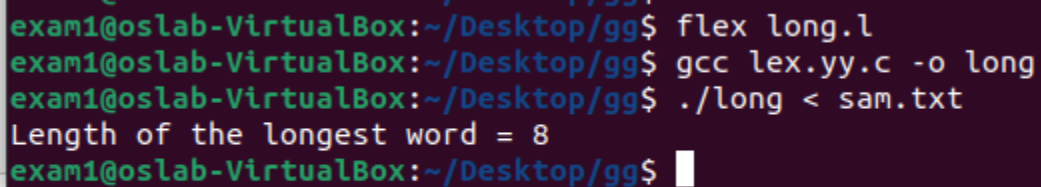
.\n    ; /* Ignore everything else */

%%

int main() {
    yylex();
    printf("Length of the longest word = %d\n", maxLen);
    return 0;
}

int yywrap() {
    return 1;
}
```

OUTPUT



```
exam1@oslab-VirtualBox:~/Desktop/gg$ flex long.l
exam1@oslab-VirtualBox:~/Desktop/gg$ gcc lex.yy.c -o long
exam1@oslab-VirtualBox:~/Desktop/gg$ ./long < sam.txt
Length of the longest word = 8
exam1@oslab-VirtualBox:~/Desktop/gg$
```

```

exam1@oslab-VirtualBox:~/Desktop/gg$ flex long.l
exam1@oslab-VirtualBox:~/Desktop/gg$ gcc lex.yy.c -o long
exam1@oslab-VirtualBox:~/Desktop/gg$ ./long < sam.txt
Length of the longest word = 8
exam1@oslab-VirtualBox:~/Desktop/gg$ flex long.l
exam1@oslab-VirtualBox:~/Desktop/gg$ gcc lex.yy.c -o long
^[[Aexam1@oslab-VirtualBox:~/Desktop/gg$ ./long < sam.txt
Longest word : compiler
Length      : 8
exam1@oslab-VirtualBox:~/Desktop/gg$ S

```

```

%{
#include <stdio.h>
#include <string.h>

int maxLen = 0;
char longest[100];
%}

%%

[a-zA-Z]+ {
    int len = strlen(yytext);
    if (len > maxLen) {
        maxLen = len;
        strcpy(longest, yytext);
    }
}

.\n ; /* Ignore everything else */

%%

int main() {
    yylex();
    printf("Longest word : %s\n", longest);
    printf("Length      : %d\n", maxLen);
    return 0;
}

```

```
int yywrap() {  
    return 1;  
}
```

```
flex longest_word.l  
gcc lex.yy.c -o longest  
./longest < sample.txt
```

CONCLUSION

The LEX program scans all words in the input and determines the length of the longest word using pattern matching and length comparison.